FACULTY
OF ELECTRICAL
ENGINEERING
CTU IN PRAGUE

B4M36SMU

Project 1 - Reinforcement Learning

20.03.2025

Tomáš Kysela

1. State proposals

1.1. Sum of player's hand, Dealer's card, Ace

In this state representation we use three values. The first value is the sum of the player's hand, the second is the dealer's face-up card, and the third is a Boolean indicating if the player has a usable Ace (counts as 11). We can calculate number of states using Equation 1.

$$\begin{aligned}\text{Hand sum} \times \text{Dealer's card} \times \text{Ace existence} = \\ 18 \times 10 \times 2 = 360\end{aligned} \tag{1}$$

1.2. Player's hand composition, Dealer's card

Unlike in Proposition 1.1, here we store exact hand composition, most likely using number of aces, number of face cards. This allows for better decision making based on hand composition, since we can better predict deck composition. Number of states is calculated using Equation 2.

$$\begin{aligned}\text{Number of aces} \times \text{Number of face cards} \times \\ \text{Number of other cards} \times \text{Dealer's card} = \\ 5 \times 3 \times 508 \times 10 = 76200\end{aligned} \tag{2}$$

1.3. Sum of player's hand, Number of cards, Dealer's card

This representation balances detail and efficiency, capturing more information than Proposition 1.1 while avoiding the complexity of Proposition 1.2. We know if we used multiple small cards or few bigger cards to get to this sum. Number of resulting states is calculated in Equation 3.

$$\begin{aligned}\text{Hand sum} \times \text{Number of cards} \times \text{Dealer's card} = \\ 18 \times 7 \times 10 = 1260\end{aligned} \tag{3}$$

2. Chosen state representation

In my implementation I chose to use state representation as proposed in Proposition 1.3. To reduce state space, merge of all states with value of player's hand of 10 and lower are merged, since here, we always want to *HIT*. Also for ease of coding, states with value of player's hand of 22 and higher are merged, since this is always a bust and it's Q/U value is never updated.

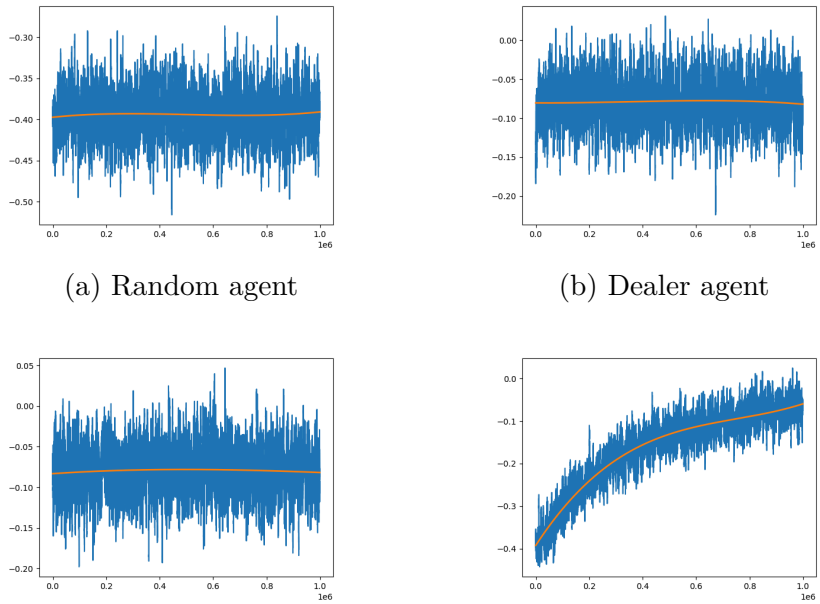
With all these simplifications, the state space is small enough, that we can use exact methods without needing too much time to finish.

3. Evaluation of agents

In this section we compare multiple agents. Most of these agents could run with much lower number of epochs, but for ease of comparison and same time axis

on graphs, we used $1 \cdot 10^6$ epochs, which was the highest number of optimal numbers of epochs for these agents.

First possible comparison between agents is using rewards. For this I used moving average of length 1000, resulting in graphs as shown in Figure 1. Here we can see that SARSA agent is clearly learning, but for comparison between other agents this is quite impractical. For that I fitted third degree polynom to each reward progression resulting in comparison shown in Figure 2. From this graph it's quite clear, that our epsilon-greedy policy is working, since overtime the SARSA agent moves from rewards of an random policy to at least dealer policy rewards, possibly better.



(a) Random agent

(b) Dealer agent

(c) TD agent

(d) SARSA agent

Figure 1: Rewards over epochs

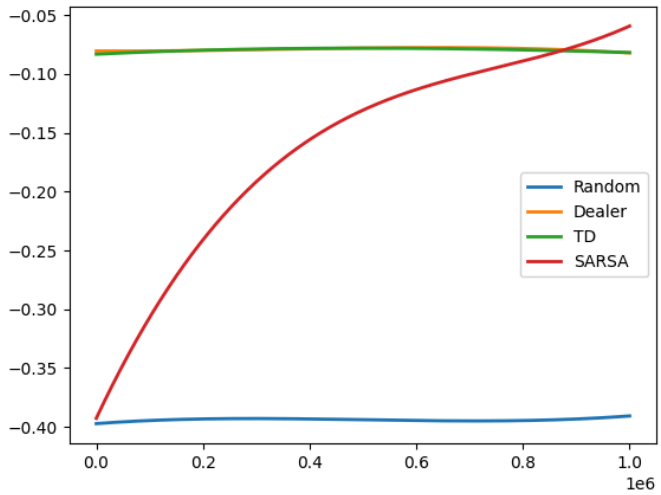
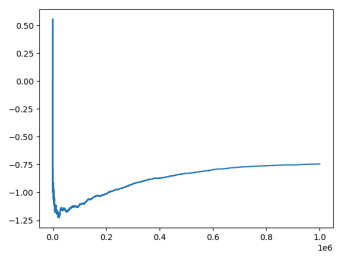
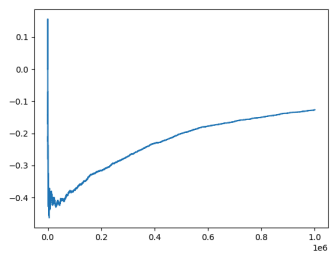


Figure 2: Comparison of rewards using 3rd degree polynoms

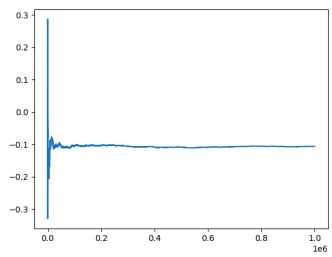
To verify my agents learn and converge we can look at state with sum of player's hand 10 and lower. Progression of U and Q values are shown in Figure 3. These graphs show, that all values converge, U values learned by TD agent significantly faster. This also shows, that Q value for action *HIT* converges to higher score than Q value for *STAND* action, which is expected, since we always want to *HIT* if we have lower sum than 11.



(a) Q values for *STAND* action learned by SARSA agent



(b) Q values for *HIT* action learned by SARSA agent



(c) U values learned by TD agent

Figure 3: Q/U values for state (10, 0, 0)